National Textile University

Department of Computer Science

Subject:

Operating System

Submitted to:

Nasir Mahmood

Submitted by:

Afia Maham

Reg number:

23-NTU-CS-1126

Lab no:

Hw - 1

Semester:

5th

Part 1: Semaphore theory

1. A counting semaphore is initialized to 7. If 10 wait() and 4 signal() operations are performed, find the final value of the semaphore.

Initial S = 7
After 10 wait() ==> 7 - 10 = -3
After 4 signal() ==> -3 + 4 = 1
Final value = 1

2. A semaphore starts with value 3. If 5 wait() and 6 signal() operations occur, calculate the resulting semaphore value.

Initial S = 3
After 5 wait() ==> 3 - 5 = -2
After 6 signal() ==> -2 + 6 = 4
Final value = 4

3. A semaphore is initialized to 0. If 8 signal() followed by 3 wait() operations are executed, find the final value.

Initial S = 0
After 8 signal() ==> 0 + 8 = 8
After 3 wait() ==> 8 - 3 = 5
Final value = 5

4. A semaphore is initialized to 2. If 5 wait() operations are executed: a) How many processes enter the critical section? b) How many processes are blocked?

Initial S = 2
After 5 wait() operations ==> 2 - 5 = -3
So, 2 will enter critical section while 3 will get blocked.
a) Processes entered = 2
b) Processes blocked = 3

5. A semaphore starts at 1. If 3 wait() and 1 signal() operations are performed: a) How many processes remain blocked? b) What is the final semaphore value?

Initial S = 1
After 3 wait() ==> 1 - 3 = -2
After 1 signal() ==> -2 + 1 = -1
a) Blocked processes = 1
b) Final value = -1

6. semaphore S = 3; wait(S); wait(S); signal(S); wait(S); wait(S); a) How many processes enter the critical section? b) What is the final value of S?

S = 3
Wait ==> 2
wait ==> 1
signal ==> 2
wait ==> 1
wait ==> 0
a) Processes entered = 4
b) Final value of S = 0

7. semaphore S = 1; wait(S); wait(S); signal(S); signal(S); a) How many processes are blocked? b) What is the final value of S?

S = 1
wait ==> 0
wait ==> −1
signal ==> 0
signal ==> 1
a) Blocked processes = 1
b) Final value = 1

8. A binary semaphore is initialized to 1. Five wait() operations are executed without any signal(). How many processes enter the critical section and how many are blocked?

Binary semaphore S = 1
5 wait() without signal

- 1 enters
- 4 blocked

Entered = 1, Blocked = 4

9. A counting semaphore is initialized to 4. If 6 processes execute wait() simultaneously, how many proceed and how many are blocked?

Counting semaphore S = 4
6 simultaneous wait()

- 4 proceed
- 2 blocked

10. A semaphore S is initialized to 2. wait(S); wait(S); wait(S); signal(S); signal(S); wait(S);
a) Track the semaphore value after each operation. b) How many processes were blocked at any time?

Initial S = 2

Operation	S value
wait	1
wait	0
wait	−1
signal	0
signal	1
wait	0

Maximum blocked at any time = 1

11. A semaphore is initialized to 0. Three processes execute wait() before any signal(). Later, 5 signal() operations are executed. a) How many processes wake up? b) What is the final semaphore value?

Initial S = 0
3 wait() ==> S = −3
5 signal() ==> −3 + 5 = 2
a) Processes awakened = 3
b) Final semaphore value = 2

Part 2: Semaphore Coding

a) **Rewritten source code**

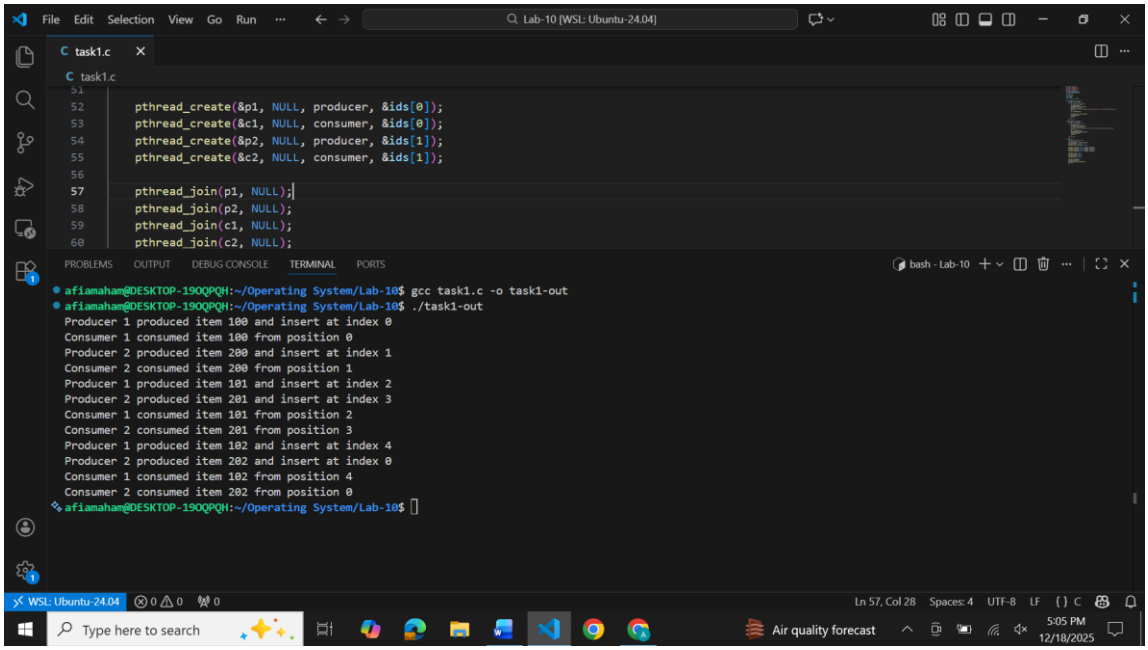
```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
   int id = *(int*)arg;
   for(int i = 0; i < 3; i++) {
      int item = id * 100 + i;
      sem_wait(&empty);
      pthread_mutex_lock(&mutex);
      buffer[in] = item;
      printf("Producer %d produced item %d and insert at index %d\n",id, item, in);
      in += 1;
      in %= BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
        }
    return NULL;
    }
void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 3; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer %d consumed item %d from position %d\n",id, item, out);
        out += 1;
        out %= BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
        }
    return NULL;
    }
int main() {
    pthread_t p1, p2, c1, c2;
    int ids[2] = {1, 2};
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p1, NULL, producer, &ids[0]);
    pthread_create(&c1, NULL, consumer, &ids[0]);
    pthread_create(&p2, NULL, producer, &ids[1]);
    pthread_create(&c2, NULL, consumer, &ids[1]);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    pthread_join(c1, NULL);
    pthread_join(c2, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
    }
```

b) Description

This program uses semaphores and a mutex to implement the Producer Consumer problem. Threads of producer and consumer both share a fixed-size buffer. The full semaphore counts the filled slots and the empty semaphore counts the empty slots in the buffer. Producer wait until a slot is empty. Then, it lock the buffer and add an item. After that, it unlocks the buffer. A consumer will wait for the slot to be filled and lock the buffer. Take an item and unlock the buffer.

c) Screenshot



Part 3: RAG (Recourse Allocation Graph)

a) Allocation Matrix

Process	A	B	C	D
P0	1	0	0	1
P1	1	0	0	0
P2	0	1	0	0
P3	0	0	0	1
P4	0	0	0	0

b) Request Matrix

Process	A	B	C	D
P0	0	0	1	0
P1	0	0	1	0
P2	1	0	0	0
P3	0	1	0	0
P4	0	0	0	1

Part 4: Banker's Algorithm

Total Existing Resources

A	B	C	D
6	4	4	2

Allocation Matrix

Process	A	B	C	D
P0	2	0	1	1
P1	1	1	0	0
P2	1	0	1	0
P3	0	1	0	1

Total Allocated Resources

- **A:** 2 + 1 + 1 + 0 = **4**
- **B:** 0 + 1 + 0 + 1 = **2**
- **C:** 1 + 0 + 1 + 0 = **2**
- **D:** 1 + 0 + 0 + 1 = **2**

Available = Total - Allocated

Available = 6 – 4, 4 – 2, 4 – 2, 2 – 2 = (2, 2, 2, 0)

Max Matrix

Process	A	B	C	D
P0	3	2	1	1
P1	1	2	0	2
P2	3	2	1	0
P3	2	1	0	1

Need Matrix

Need = Max – Allocation

Process	A	B	C	D
P0	1	2	0	0
P1	0	1	0	2
P2	2	2	0	0
P3	2	0	0	0

Step-by-step Process and result: